

56.合并区间

合并区间

1.问题

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (51.29%)	2561	0	-	-	0

- ▼ Tags
- ▼ Companies

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [starti, endi]` 。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例 1:

输入: intervals = [[1,3],[2,6],[8,10],[15,18]]

输出: [[1,6],[8,10],[15,18]]

解释: 区间 [1,3] 和 [2,6] 重叠, 将它们合并为 [1,6].

示例 2:

输入: intervals = [[1,4],[4,5]]

输出: [[1,5]]

解释: 区间 [1,4] 和 [4,5] 可被视为重叠区间。

提示:

- `1 <= intervals.length <= 104`
- `intervals[i].length == 2`
- `0 <= starti <= endi <= 104`

[Discussion](#) | [Solution](#)

2.解决方法

好的，我们来分析一下你的想法和这个问题（LeetCode 56 合并区间）。

你的想法：使用 `HashMap` 标记数字

1. 思路:
- 创建一个 `HashMap`（比如 `Map<Integer, Boolean>` 或 `Map<Integer, Integer>`）。
 - 遍历输入的 `intervals` 数组中的每一个区间 `[start, end]`。
 - 对于 `start` 到 `end` 之间的每一个整数 `num`，在 `HashMap` 中标记它，例如 `map.put(num, true)`。
 - 处理完所有区间后，遍历 `HashMap` 中的键（这些是被覆盖的数字）。由于 `HashMap` 本身不保证顺序，你需要先将键排序。
 - 遍历排序后的数字，找到连续的数字块，根据这些连续块的起点和终点，构建出合并后的区间。
2. 潜在问题:

- **内存消耗：** 如果区间的范围非常大，例如 `[[0, 10000]]`，你的 `HashMap` 需要存储 10001 个条目。如果有很多这样的区间或者区间跨度很大，内存消耗会非常巨大，甚至可能导致内存溢出。
- **时间效率：**
 - 填充 `HashMap` 的时间取决于所有区间覆盖的点的总数，而不是区间的数量。如果区间很大，这一步会很慢。
 - 从 `HashMap` 中提取键并排序需要 $O(N \log N)$ 的时间，其中 N 是被覆盖的点的总数， N 可能远大于区间的数量 n 。
 - 最后根据排序后的点重建区间也需要遍历 N 个点。
- **实现复杂度：** 从排序后的离散点重建连续区间逻辑稍微有点绕。

我的想法（标准解法）：排序 + 合并

这通常是解决这类问题的更高效、更常用的方法：

1. **排序：** 这是关键一步。将所有区间按照**起始点**从小到大排序。如果起始点相同，可以按结束点排序（虽然对这个特定问题不强制要求）。
 - 例如：`[[1,3],[8,10],[2,6],[15,18]]` 排序后变成 `[[1,3],[2,6],[8,10],[15,18]]`。
2. **遍历与合并：**
 - 创建一个列表（比如 `ArrayList<int[]>`）用于存放合并后的结果 `merged`。
 - 将排序后的第一个区间加入 `merged` 列表。
 - 从第二个区间开始，遍历排序后的区间列表：
 - 取出当前要处理的区间 `currentInterval`。
 - 取出 `merged` 列表中**最后一个**区间 `lastMergedInterval`。
 - **判断是否重叠：** 如果 `currentInterval` 的起始点 `currentInterval[0]` **小于或等于** `lastMergedInterval` 的结束点 `lastMergedInterval[1]`，说明它们有重叠。
 - **合并操作：** 如果有重叠，将 `lastMergedInterval` 的结束点更新为它自身结束点和 `currentInterval` 结束点中的**较大值**。即 `lastMergedInterval[1] = Math.max(lastMergedInterval[1], currentInterval[1])`。这样就将 `currentInterval` 合并进了 `lastMergedInterval`。
 - **无重叠：** 如果 `currentInterval` 的起始点 **大于** `lastMergedInterval` 的结束点，说明它们不重叠，此时将 `currentInterval` 作为一个新的区间添加到 `merged` 列表的末尾。
 - 遍历结束后，`merged` 列表中就是所有合并后的区间。
3. **结果转换：** 将 `merged` 列表转换成题目要求的 `int[][]` 格式返回。

为什么排序+合并更好？

- **时间复杂度：** 主要开销在于排序，为 $O(n \log n)$ ，其中 n 是区间的数量。后续的合并遍历只需要 $O(n)$ 的时间。总时间复杂度是 $O(n \log n)$ 。
- **空间复杂度：** 主要需要 $O(n)$ 的空间来存储排序后的列表（如果原地排序则更少）和结果列表。
- **不受区间范围影响：** 这种方法的时间和空间复杂度只与区间的数量有关，与区间内数字的大小无关，更加稳定高效。

代码实现（排序+合并）：

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

class Solution {
    public int[][] merge(int[][] intervals) {
```

```

// 1. 处理边界情况
if (intervals == null || intervals.length <= 1) {
    return intervals;
}

// 2. 按照区间的起始点排序
// Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
// 或者使用 Comparator.comparingInt
Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

// 3. 遍历并合并
List<int[]> merged = new ArrayList<>();
// 先将第一个区间加入结果列表
merged.add(intervals[0]);

for (int i = 1; i < intervals.length; i++) {
    int[] currentInterval = intervals[i];
    // 获取结果列表中最后一个区间
    int[] lastMergedInterval = merged.get(merged.size() - 1);

    // 检查是否重叠: 当前区间的 start <= 上一个合并区间的 end
    if (currentInterval[0] <= lastMergedInterval[1]) {
        // 有重叠, 合并区间, 更新上一个合并区间的 end
        // end 取两者 end 的最大值
        lastMergedInterval[1] = Math.max(lastMergedInterval[1], currentInterval[1]);
    } else {
        // 没有重叠, 将当前区间作为一个新的合并区间加入结果列表
        merged.add(currentInterval);
    }
}

// 4. 将 List<int[]> 转换为 int[][]
return merged.toArray(new int[merged.size()][]);
}
}

```

总结来说, 你的 HashMap 思路虽然直观, 但在效率和内存方面有较大限制。排序+合并的方法是解决此类区间问题的标准且高效的策略。

3. 示例演示

假设输入 `intervals = [[1,4],[0,2],[3,5]]` (<http://vscodecontentref/0>)

1. **排序:** 按起始点排序后得到 `[[0,2], [1,4], [3,5]]`。

2. **合并过程:**

- 初始化 `merged` 列表。
- 将第一个区间 `[0,2]` 加入 `merged`。 `merged = [[0, 2]]`
- 处理下一个区间 `current = [1,4]`。
 - `last = [0, 2]`。
 - 检查重叠: `current[0] (1) <= last[1] (2)`? 是, 重叠。
 - 合并: 更新 `last[1]` 为 `max(last[1], current[1]) = max(2, 4) = 4`。

- `merged` 变为 `[[0, 4]]`
 - 处理下一个区间 `current = [3, 5]`。
 - `last = [0, 4]`。
 - 检查重叠: `current[0] (3) <= last[1] (4)`? 是, 重叠。
 - 合并: 更新 `last[1]` 为 `max(last[1], current[1]) = max(4, 5) = 5`。
 - `merged` 变为 `[[0, 5]]`
 - 所有区间处理完毕。
3. 返回结果: `[[0, 5]]`

5. 如何在面试中讲解

当面试官问到这题时, 你可以这样组织思路:

1. **确认问题:** "好的, 这道题的目标是合并所有重叠的区间。比如输入 `[[1,3],[2,6],[8,10]]`, `[1,3]` 和 `[2,6]` 重叠, 需要合并成 `[1,6]`, 最终结果是 `[[1,6],[8,10]]`。" (确保你理解了输入输出和重叠的定义)
2. **初步想法 (可选):** "一个暴力的方法是比较每对区间, 看它们是否重叠, 如果重叠就合并, 然后重复这个过程直到没有区间可以合并。但这样效率不高, 可能是 $O(n^2)$ 甚至更高。" (展示你考虑过简单但不优的方法)
3. **核心思路 - 排序:** "一个更高效的方法是利用排序。如果我们**首先按照区间的起始点对所有区间进行排序**, 那么我们就可以按顺序处理它们。这样, 一个区间只需要和它紧邻的或者说结果集中最后一个区间比较是否重叠。"
4. **合并逻辑:** "排序后, 我们创建一个列表来存放合并后的结果。先把第一个排序后的区间放进去。然后, 我们遍历剩下的排序后的区间。对于每个当前区间, 我们看它和结果列表里**最后一个**区间是否重叠。"
5. **判断与操作:** "判断重叠的条件是: 当前区间的起始点小于等于结果列表最后一个区间的结束点。
 - 如果**重叠**, 我们就更新结果列表最后一个区间的结束点, 把它扩展成两个区间结束点的最大值, 相当于把当前区间合并进去了。
 - 如果**不重叠**, 说明遇到了一个新的、独立的区间段, 我们就把当前区间直接添加到结果列表的末尾。"
6. **复杂度分析:** "这个算法的主要时间开销在于排序, 所以时间复杂度是 $O(n \log n)$, n 是区间的数量。遍历合并的过程是 $O(n)$ 。空间复杂度主要是存储结果列表, 是 $O(n)$ 。"
7. **总结与示例 (可选):** "总的来说, 就是'排序 → 遍历 → 比较 → 合并或添加'的流程。如果需要, 我可以演示一个例子, 比如 `[[1,4],[0,2],[3,5]] ...`"